Développement Mobile avec Dart & Flutter

Partie 3/10 : Installation et environnement de développement Flutter

Pr. Abdelfattah Abassi

Faculté des Sciences et Techniques de Settat
Université Hassan I

Laboratoire LAVETE
Département de Physique Appliquée

Année universitaire 2026–2027

1.1 Introduction à l'écosystème de développement

Qu'est-ce qu'un environnement de développement Flutter ?

C'est l'ensemble des outils logiciels interconnectés nécessaires pour écrire, compiler, exécuter et déboguer une application Flutter.

- **Le SDK Flutter :** le framework, les bibliothèques, les outils et le moteur de rendu.
- **Le SDK Dart :** le langage de programmation utilisé par Flutter.
- **L'éditeur / IDE :** VS Code ou Android Studio pour écrire et organiser le code.
- **Les cibles d'exécution :** émulateurs, navigateurs Web ou appareils physiques.

1.2 Flutter SDK vs Dart SDK

Flutter SDK

- Contient le framework Flutter.
- Fournit la bibliothèque de widgets.
- Fournit l'outil en ligne de commande `flutter`.
- Intègre le moteur graphique.
- Gère le processus de compilation multiplateforme.

Dart SDK

- Contient le langage Dart.
- Fournit le compilateur Dart.
- Fournit les bibliothèques `dart:core`, `dart:async`, etc.
- Fournit le gestionnaire de paquets `pub`.
- Est fourni avec le SDK Flutter.

1.3 Choix de l'IDE : Android Studio vs VS Code

Critère	Android Studio	Visual Studio Code
Empreinte mémoire	Plus importante	Plus légère
Configuration	Solution intégrée	Nécessite des extensions
Émulateurs	Gestion intégrée des AVD	Utilise les outils Android
Démarrage	Plus lent	Généralement plus rapide
Utilisation	Adapté aux machines puissantes	Très pratique pour les TPs

Pour les TPs

Pour des machines limitées en ressources, VS Code + Chrome peut être une solution plus légère pour les premiers TPs.

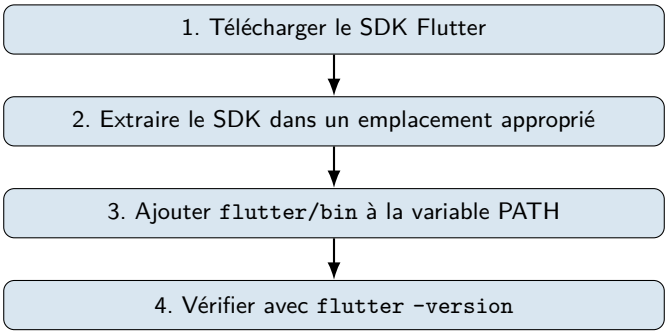
1.4 Cibles d'exécution et outils légers

- **Émulateur Android :** smartphone virtuel exécuté sur l'ordinateur. Il peut consommer beaucoup de CPU et de RAM.
- **Navigateur Chrome :** permet d'exécuter une application Flutter Web sans utiliser immédiatement un émulateur Android.
- **DartPad :** environnement Web permettant de tester rapidement du code Dart et certains exemples Flutter.

Stratégie recommandée

Commencer par DartPad et Flutter Web avant de passer à l'émulateur Android.

2.1 Étapes d'installation globale



2.2 Téléchargement et extraction du SDK

Où installer Flutter ?

Éviter les dossiers nécessitant des privilèges administrateur.

- **Windows :**
  `C:\src\flutter`
  ou
  `C:\dev\flutter`
- **Linux / macOS :**
  `~/development/flutter`
  ou
  `~/flutter`

2.3 Configuration du PATH sous Windows

La variable **PATH** permet au terminal de reconnaître la commande `flutter` depuis n'importe quel répertoire.

Procédure

1. Rechercher : **Modifier les variables d'environnement système**.
2. Cliquer sur : **Variables d'environnement**.
3. Dans les variables utilisateur, sélectionner `Path`.
4. Cliquer sur **Modifier**.
5. Ajouter le chemin :

```
C:\src\flutter\bin
```

2.4 Configuration du PATH sous macOS et Linux

Sous macOS et Linux, le PATH peut être configuré dans le fichier du shell utilisé.

Exemple

```
export PATH="$HOME/development/flutter/bin:$PATH"
```

Pour rendre la modification permanente :

- ajouter la commande dans `~/.bashrc` ou `~/.zshrc` ;
- recharger la configuration :
  `source ~/.bashrc`
  ou
  `source ~/.zshrc`

2.5 Vérification du SDK Flutter

Ouvrir un nouveau terminal puis exécuter :

Commande

```
flutter --version
```

Exemple de sortie :

```
Flutter 3.x.x
channel stable
Framework  revision ...
Engine  revision ...
Tools  Dart 3.x.x  DevTools ...
```

Remarque

La version affichée dépend de la version de Flutter installée.

3.1 Vérification et relation avec le SDK Dart

Dart est fourni avec Flutter.

Commande

```
dart --version
```

```
Dart SDK version: 3.x.x
```

À retenir

Il n'est généralement pas nécessaire d'installer séparément Dart pour développer des applications Flutter.

4.1 Présentation de `flutter doctor`

L'outil de diagnostic

`flutter doctor` analyse l'environnement de développement et signale les composants correctement configurés ou manquants.

Commande

```
flutter doctor
```

- ✓ : composant correctement configuré.
- **!** : avertissement.
- **X** : problème nécessitant une intervention.

4.2 Interprétation d'un rapport `flutter doctor`

```
Doctor summary:
[] Flutter
[] Android toolchain
[] Chrome
[] Android Studio
[] VS Code
[] Connected device
[] Network resources

No issues found!
```

Objectif

Avant de commencer un TP Flutter, il est recommandé de vérifier l'environnement avec :

`flutter doctor`

4.3 Problème : “Flutter non trouvé”

Symptôme sous Windows

```
'flutter' n'est pas reconnu en tant que commande interne ou externe.
```

Causes possibles :

1. Flutter n'est pas ajouté au PATH.
2. Le terminal n'a pas été redémarré.
3. Le chemin vers Flutter est incorrect.
4. Le dossier `bin` n'a pas été ajouté au PATH.

Vérification

Sous Windows :

```
where flutter
```

4.4 Android SDK et licences

Message possible

```
Android licenses not accepted
```

Solution :

1. Ouvrir un terminal.
2. Exécuter :

```
flutter doctor -android-licenses
```

3. Lire les licences proposées.
4. Répondre y lorsque cela est nécessaire.

5.1 Lister les appareils disponibles

La commande `flutter devices` permet de connaître les appareils et plateformes disponibles.

Commande

```
flutter devices
```

Exemple :

```
2 connected devices:

Chrome
Windows
```

5.2 Types de périphériques gérés

- **Périphérique physique :** smartphone connecté à l'ordinateur.
- **Émulateur Android :** appareil Android virtuel.
- **Navigateur Web :** Chrome ou Edge.
- **Desktop :** Windows, macOS ou Linux.

Commande utile

```
flutter devices
```

6.1 Configuration d'Android Studio

Android Studio permet notamment d'installer et de gérer le SDK Android et les émulateurs.

1. Installer Android Studio.
2. Lancer le **Setup Wizard**.
3. Installer les composants Android nécessaires.
4. Vérifier le SDK Android.

Important

Android Studio n'est pas obligatoire comme éditeur de code, mais ses outils Android sont très utiles pour le développement Flutter Android.

6.2 Utilisation du SDK Manager

Dans Android Studio :

Settings / Preferences → Android SDK

SDK Platforms :

- Installer une plateforme Android compatible.

SDK Tools :

- `Android SDK Build-Tools`
- `Android Emulator`
- `Android SDK Platform-Tools`
- `Android SDK Command-line Tools`

6.3 Création d'un émulateur Android

Via le **Device Manager** :

1. Cliquer sur **Create Device**.
2. Choisir un profil matériel.
3. Sélectionner une image système Android.
4. Télécharger l'image si nécessaire.
5. Créer l'AVD.
6. Démarrer l'émulateur.

7.1 Configuration de VS Code

Préparation

1. Installer **Visual Studio Code**.
2. Ouvrir le panneau des extensions :
   `Ctrl+Shift+X`

Extensions :

- **Flutter**
- **Dart**

Conseil

L'installation de l'extension Flutter permet généralement d'obtenir également le support Dart nécessaire.

7.2 Utilisation de VS Code pour Flutter

- **Palette de commandes :**
  `Ctrl+Shift+P`
- **Terminal intégré :**
  Menu *Terminal* → *New Terminal*
- **Sélection du périphérique :**
  Cliquer sur le périphérique affiché dans la barre d'état.

Raccourci important

`Ctrl+Shift+P` permet d'accéder rapidement aux commandes Flutter.

8.1 Présentation de DartPad

Un environnement Web pour Dart

DartPad permet d'écrire et d'exécuter du code Dart directement dans un navigateur.

`https://dartpad.dev`

Avantages

- Pas d'installation.
- Accessible depuis un navigateur.
- Idéal pour apprendre Dart.
- Exécution rapide.

Limites

- Ne remplace pas un projet Flutter complet.
- Accès limité aux ressources locales.
- Pas adapté aux fonctionnalités natives complexes.

9.1 Exécution sur le Web pour l'apprentissage

Pour les étudiants disposant d'ordinateurs peu puissants, un émulateur Android peut être relativement lourd.

Alternative : Chrome

```
flutter run -d chrome
```

- Pas besoin de démarrer un émulateur Android.
- Démarrage généralement rapide.
- Hot Reload disponible.
- Très pratique pour les premiers TP.

10.1 Création du premier projet

Ouvrir un terminal et exécuter :

Commande

```
flutter create hello_flutter
```

- Flutter génère automatiquement l'arborescence du projet.
- Le nom du projet doit respecter les conventions de nommage Dart.
- Exemple :
  `hello_flutter`

10.2 Navigation et exécution

Étape 1 : accéder au projet

```
cd hello_flutter
```

Étape 2 : exécuter le projet

```
flutter run
```

Résultat

Si une cible compatible est disponible, Flutter compile et lance l'application.

11.1 Scénarios d'erreurs et solutions

1. No connected devices found

Vérifier les périphériques disponibles :

```
flutter devices
```

Ou lancer Chrome :

```
flutter run -d chrome
```

2. Android SDK introuvable

Configurer le chemin du SDK Android :

```
flutter config -android-sdk "chemin/vers/android/sdk"
```

3. Projet ou cache corrompu

Exécuter :

```
flutter clean
```

puis

```
flutter pub get
```

12.1 Exercices pratiques

Exercice 1 : Diagnostic

- Exécuter : `flutter doctor -v`
- Identifier les composants correctement configurés.

Exercice 2 : Première fonction en Dart

- Ouvrir **DartPad** (`https://dartpad.dev`).
- Écrire une fonction nommée `somme` qui prend deux nombres (`double` ou `int`) en paramètres et retourne leur somme.
- Appeler cette fonction dans la méthode `main()` et afficher le résultat avec `print()`.

12.2 Solution : Exemple de code Dart

Exemple de solution dans DartPad

```dart
double calculerSomme(double a, double b) {
    return a + b;
}

void main() {
    double x = 5.5;
    double y = 4.5;

    // Appel de la fonction
    double resultat = calculerSomme(x, y);

    // Affichage du resultat
    print("La somme de $x et $y est : $resultat");
}
```

Résultat dans la console

```
La somme de 5.5 et 4.5 est : 10.0
```